

# Python for Artificial Intelligence

Module 1: Computational Foundations

---

## Problem Set 2

Boolean Logic and Conditional Control Flow

*Lab Project: The AI Weather-Bot 3000*

---

|                    |                                                                                                 |
|--------------------|-------------------------------------------------------------------------------------------------|
| <b>Course</b>      | Python for AI (Module 1)<br>introductory computing practicals                                   |
| <b>Author</b>      | Dr. Prateek, Assistant Professor, School of AI                                                  |
| <b>Format</b>      | In-class supervised Practical (2 hours), with<br>one week allowed for write-up and resubmission |
| <b>Total Marks</b> | 100 (+10 Bonus)                                                                                 |

Read this document in full before you begin writing code.

## Contents

|          |                                                      |          |
|----------|------------------------------------------------------|----------|
| <b>1</b> | <b>Objective</b>                                     | <b>2</b> |
| <b>2</b> | <b>Practical Format</b>                              | <b>2</b> |
| <b>3</b> | <b>Task 1: Boolean Experiments (30 minutes)</b>      | <b>2</b> |
| 3.1      | Discussion Points . . . . .                          | 2        |
| <b>4</b> | <b>Task 2: Building the Weather-Bot (90 minutes)</b> | <b>3</b> |
| 4.1      | Project Requirements . . . . .                       | 3        |
| 4.2      | Example Lab Code Structure . . . . .                 | 3        |
| <b>5</b> | <b>Edge Cases for Consideration</b>                  | <b>4</b> |
| <b>6</b> | <b>Deliverables</b>                                  | <b>4</b> |
| <b>7</b> | <b>Learning Outcomes</b>                             | <b>4</b> |
|          | <b>Academic Integrity</b>                            | <b>5</b> |

## 1 Objective

This Practical introduces **Boolean logic** and **conditional control flow** — the mechanisms by which a program makes decisions. Having already worked with data types, casting, and string manipulation in Problem Set 1, you will now combine those skills with comparison and logical operators to build a program that reasons about its input and responds differently depending on what it receives.

By the end of this Practical you should be able to:

- (i) evaluate and predict the result of comparison and logical expressions,
- (ii) write syntactically correct `if/elif/else` chains, respecting Python's colon-and-indentation rules, and
- (iii) combine numeric and string-based conditions within a single coherent program.

## 2 Practical Format

This is a two-hour, in-class, supervised Practical session, structured as follows:

| Task   | Description                             | Duration   |
|--------|-----------------------------------------|------------|
| Task 1 | Boolean Experiments (interactive shell) | 30 minutes |
| Task 2 | Building the Weather-Bot (script file)  | 90 minutes |

Although the Practical is time-boxed to two hours in class, you have **one week** from the date of issue to finalize, test, and submit your completed work, as with Problem Set 1.

## 3 Task 1: Boolean Experiments (30 minutes)

Open your Python interactive shell (the REPL). For each expression below, **write down your predicted output before pressing Enter**. This is not optional: the value of this exercise lies in confronting your assumptions, not in seeing the correct answer.

```
1 >>> 42 == 42
2 >>> 'AI' == 'ai'
3 >>> 10 > 5 and 2 < 1
```

### 3.1 Discussion Points

- `==` tests *equality* and always returns a Boolean (`True` or `False`); it must not be confused with the assignment operator `=`.
- String comparison in Python is **case-sensitive**. The strings `"AI"` and `"ai"` are different objects as far as `==` is concerned, because the underlying character codes differ. If you want a case-insensitive comparison, you must explicitly normalise both sides, e.g. with `.lower()`.
- The **and** operator returns `True` only if *both* operands are `True`. Python evaluates such expressions left to right and stops as soon as the result is determined (*short-circuit evaluation*) — for `10 > 5 and 2 < 1`, the first comparison is `True`, so Python must still check the second, which is `False`; the overall expression is therefore `False`.

Record your three predictions and the actual shell output in your lab notebook or submission write-up, along with a one-line explanation for any prediction you got wrong.

## 4 Task 2: Building the Weather-Bot (90 minutes)

Create a new file in your editor and write a multi-line Python program named `weather_bot.py`. Unlike Task 1, this code will be saved and executed as a script, not typed line-by-line into the shell.

### 4.1 Project Requirements

1. **Input.** Use `input()` to ask the user for the current temperature, in degrees Celsius.
2. **Casting.** Convert the input string to an integer using `int()` so that numerical comparisons can be performed. As in Problem Set 1, validate the string with `str.isdigit()` before converting it, and handle the case where the input is not a valid number.
3. **Logic.** Build an `if/elif/else` chain using the following thresholds:
  - If temperature  $> 30$ : *"It's hot! AI suggests turning on the AC."*
  - If temperature  $< 15$ : *"Chilly! AI suggests a jacket."*
  - Otherwise: *"Temperature is optimal. Enjoy your day!"*
4. **Bonus Feature.** Ask the user a second question, e.g. *"Is it raining?"*, and store the response as a string. Use a membership test of the form `if "yes" in rain_check.lower():` to decide whether to suggest carrying an umbrella. Using `in` rather than `==` allows the check to match answers such as "Yes", "yes please", or "YES" without requiring an exact match.

### 4.2 Example Lab Code Structure

The listing below illustrates the expected shape of your program. You are required to extend it, not merely copy it: in particular, you must add the `isdigit()` validation described above, which is intentionally omitted here.

```

1  """
2  Program : Weather-Bot 3000
3  Purpose : Gives simple lifestyle advice based on temperature and rain
4
5  Author  : <Your Full Name>
6  Date    : <Submission Date>
7  """
8  print("Welcome to your AI Climate Assistant.")
9  temp_input = input("What is the current temperature in Celsius? ")
10
11 # Step 1: Convert input to an integer
12 temperature = int(temp_input)
13
14 # Step 2: Decision logic based on temperature
15 if temperature > 30:
16     print("AI Alert: It's hot! AI suggests turning on the AC.")
17 elif temperature < 15:
18     print("AI Alert: Chilly! AI suggests a jacket.")
19 else:
20     print("AI Analysis: Temperature is optimal. Enjoy your day!")
21
22 # Step 3: Combined logic (bonus feature)
23 rain_check = input("Is it raining outside? (yes/no) ")
24 if "yes" in rain_check.lower() and temperature < 15:

```

```
25     print("AI Recommendation: Stay indoors today , and carry an  
        umbrella.")  
26 elif "yes" in rain_check.lower():  
27     print("AI Recommendation: Carry an umbrella.")
```

*Note on the combined condition.* The final `if/elif` pair deliberately reuses the same temperature threshold ( $< 15$ ) defined in Step 2. Keeping thresholds consistent across a program, rather than introducing a second, slightly different value, is good practice: it avoids contradictory advice and makes the logic easier to reason about and to modify later.

## 5 Edge Cases for Consideration

As in Problem Set 1, consider how your program should respond to inputs its author did not anticipate. Be prepared to explain your program's behaviour for each of the following:

1. A temperature entered as a non-numeric string (e.g., "warm"), which would raise a `ValueError` if cast without validation.
2. A temperature entered with a leading or trailing space (e.g., " 25"), and whether `str.isdigit()` behaves as you expect on such input.
3. A rain response that is empty (the user presses Enter without typing anything), or one that contains "yes" as a substring of an unrelated word (e.g., "yesterday"). Consider whether this is a genuine limitation of the `in` check and how a more careful implementation might avoid it.
4. A temperature of exactly 30 or exactly 15. Trace through your `if/elif/else` chain by hand and confirm which branch executes at each boundary.

## 6 Deliverables

1. **Source Code:** A single file named `weather_bot.py`.
2. **Task 1 Write-up:** Your three predictions from the Boolean Experiments, the actual shell output, and a brief explanation for any mismatch.
3. **Console Log:** A screenshot or pasted terminal output showing at least three runs of `weather_bot.py`, covering the hot, chilly, and optimal branches respectively.

All files must be submitted through the course portal by the deadline stated on the title page. Late submissions will be handled according to the course's standard late-submission policy.

## 7 Learning Outcomes

On successful completion of this Practical, you should have:

1. mastered the colon-and-indentation rules that govern Python block structure, and that most commonly trip up beginners moving from other languages or from the interactive shell to script files;
2. practiced combining different data types (integers and strings) within a single logical flow; and

3. produced a small, self-contained “smart assistant” that demonstrates the same input-validate-decide pattern underlying far more sophisticated AI systems.

## Academic Integrity

You may discuss approaches with peers during the supervised Practical session, but the code you submit must be written independently and must reflect your own understanding. Submissions found to be substantially copied, from a peer or from an external source, will be treated in accordance with the institute’s academic integrity policy.